

Milestone 2 - Cart / Order contribution

Status: proposed design, pending team agreement. This is one member's contribution, not the complete team REPORT.pdf.

Source: supplied Mini-Amazon Project Description; repository AustinLy612/mini_amazon at bf78ef7ac982d82f3fbe6bb798f4fe2fb5df80cc.

1. Scope and milestone boundaries

Milestone 2 requires every member to run the skeleton and complete TUTORIAL.md, a shared database design, a page-by-page website design, README.txt with members/roles/progress/repository, and a team REPORT.pdf. Implementation of the cart-query endpoint is required in milestone 3, not milestone 2. Checkout and editing are final basic requirements. Optional saved-for-later and coupons are excluded from this initial design.

The current repository contains Users, Products, and Purchases only. Users lacks balance/address; Products lacks categories, descriptions, images and ownership; Purchases cannot represent quantities, sellers, fixed prices or multi-item orders. Preserve this legacy example during development; Users owner should later switch purchase history to Orders/OrderItems. Do not duplicate new orders into legacy Purchases. No historic data migration is attempted because seller/quantity/price information is missing.

2. Database design

All money uses NUMERIC(12,2), with Python Decimal for calculations. All new timestamps use TIMESTAMPTZ. Existing UTC-naive purchase timestamps remain legacy-only. A user can buy and sell. One global product price is shared by sellers, consistent with the base requirement and skeleton. No taxes, shipping charges or discounts in this first version. Self-purchases are allowed; aggregate net balance changes by account to handle a buyer who is also a seller correctly.

Users (existing; Users owner)

Existing id PK, email UNIQUE NOT NULL, password hash NOT NULL, firstname and lastname NOT NULL. Add balance NUMERIC(12,2) NOT NULL DEFAULT 0 CHECK >= 0; add nullable address TEXT for compatibility with existing registration/CSV data. Checkout requires a nonblank name and address. Account owner implements profile and balance editing. All balance modifications must use concurrency-safe transactions; withdrawal cannot create negative balance.

Products (existing; Products owner)

Existing id PK, name UNIQUE NOT NULL, price NUMERIC(12,2) NOT NULL, available BOOLEAN. Add CHECK price >= 0. Null available is treated as unavailable. Product descriptions/categories/images/creator constraints remain for the Products owner to design and include in the team report. Product deletion is restricted by references; prefer deactivation to preserve history.

Inventory (proposed; Sellers owner)

(seller_id, product_id) composite PK; both NOT NULL FKs to Users and Products. quantity INT NOT NULL CHECK >= 0; active BOOLEAN NOT NULL DEFAULT TRUE. One seller has at most one listing per product. Removing a listing sets active=false; retain the row so existing cart rows remain visible and are labelled unavailable. Inventory is decremented at checkout, never again at fulfillment.

CartItems (Carts owner)

(buyer_id, seller_id, product_id) composite PK. buyer_id FK Users; (seller_id, product_id) composite FK Inventory; quantity INT NOT NULL CHECK > 0. One cart is implicit per user; zero rows means an empty cart. Repeated additions merge quantities for the same seller/product. Different sellers stay separate. Rows persist in PostgreSQL across logout/restart. No price is stored: display current Products.price. Stock is not reserved. Quantity <= stock is a service-level check at add/update/checkout, not a cross-table CHECK; stock can later fall below a saved cart quantity.

Orders (Carts owner; read by Users and Sellers)

id identity PK; buyer_id NOT NULL FK Users; placed_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP. shipping_name and shipping_address TEXT NOT NULL CHECK nonblank store delivery snapshots. Index (buyer_id, placed_at DESC, id DESC) supports purchase history. Total and fulfillment are derived, not independently mutable columns.

OrderItems (Carts owner; fulfillment field written by Sellers)

(order_id, seller_id, product_id) composite PK, with NOT NULL FKs to Orders, Users and Products. product_name TEXT NOT NULL snapshot, quantity INT NOT NULL CHECK > 0, unit_price NUMERIC(12,2) NOT NULL CHECK >= 0; fulfilled_at nullable TIMESTAMPTZ. NULL means pending; non-NULL means fulfilled at that time. Index (seller_id, order_id) supports seller history. No FK to live Inventory: historical orders survive a listing becoming inactive. Buyer cannot edit orders. Seller can only set fulfilled_at on their own lines, once; product/quantity/price snapshots remain immutable through application routes. SQL constraints alone do not enforce this authorization.

Relationships and derived fields

Users 1:N CartItems (buyer); Inventory 1:N CartItems; Users 1:N Orders; Orders 1:N OrderItems; Users 1:N OrderItems (seller); Products 1:N Inventory and OrderItems. Order total = SUM(quantity * unit_price); item count = SUM(quantity), distinct line count = COUNT(*). Fulfilled = at least one line AND all lines have fulfilled_at. Checkout creates at least one line in the same transaction as its order header; this nonempty invariant is application-enforced. FKs default to restricted/no-action deletion, with no cascading deletion of order history.

3. Page-by-page flow

1. Product detail (Products): display sellers, available stock, quantity input and Add to cart. Guest is sent to login; successful POST returns to cart with a message. Reject noninteger/nonpositive quantities, inactive listings and insufficient stock. Same product/seller merges into the existing row. Product details are not present in the skeleton yet.
2. Cart /cart (Carts): show product, seller, quantity, live unit price, subtotal, availability and total. Planned controls: update quantity, remove line, continue shopping, submit entire cart. Empty cart shows a shopping link and no checkout control. Invalid edits retain old values and show a clear error. Unavailable lines remain visible for removal. The delivered starter currently implements display only.
3. Checkout confirmation (Carts): display current items/prices, total, account balance and delivery address before confirmation. The submit request revalidates all values. If a displayed price changed, reject that attempt and ask the buyer to review refreshed prices before confirming again. On insufficient stock/balance or missing address, retain cart, create no order and show a specific error. Successful checkout redirects to order detail.
4. Order detail /orders/<order_id> (Carts): require login and buyer ownership; another buyer receives 404. Show order number/date, delivery snapshot, each product/seller, purchased quantity, final unit price/subtotal, total, each

fulfilled_at or Pending, and overall status. No buyer edit/cancel controls.

5. Purchase history (Users): newest orders first with total, quantity count and derived fulfillment status; every summary links to order detail. Replace legacy Purchases queries when integrating.

6. Seller order list/detail (Sellers): show only current seller's order lines, their subtotals and quantities, buyer/address/date, and overall order fulfillment flag if required; do not expose other sellers' line details or totals. Fulfill POST updates only owned pending lines. Buyers see the updated state on refresh.

Main flow: product detail -> cart -> confirmation -> order detail; account -> purchase history -> order detail; seller inventory/order history -> fulfill own line -> buyer order status updates. Login returns the user to the requested page. Product/seller links are integrated once teammates supply actual routes.

4. Proposed route contract

Implemented: GET /cart (HTML) and GET /api/cart (JSON), authenticated user's own cart only.

Cart.get_items(buyer_id) is the SQL model entry point for milestone 3. Query-string buyer IDs cannot override login identity. JSON currency is a decimal string.

Planned: POST /cart/items (product_id, seller_id, quantity), POST /cart/items/<seller_id>/<product_id>/quantity, POST /cart/items/<seller_id>/<product_id>/remove, GET /checkout, POST /checkout, GET /orders/<order_id>. Seller fulfillment route belongs to Sellers. All mutations use POST, CSRF protection, server validation, parameterized SQL and owner authorization. Never accept buyer_id, final price or computed total as authoritative client input.

5. Atomic checkout algorithm (design, not yet implemented)

Use ONE with app.db.engine.begin() as conn: block and parameterized conn.execute(text(sql), params). Do not call app.db.execute repeatedly for checkout: each call commits a separate transaction.

1. Authenticate, validate CSRF and read confirmation values. Serialize per-buyer cart mutations and checkout with a transaction-scoped lock used by all cart writers. Read the nonempty cart.

2. Lock all participating Users rows in ascending id order, Products rows in ascending id order, and Inventory rows in (seller_id, product_id) order. All collaborating balance/inventory writers must adopt a compatible lock order. Re-read live prices/stock/active flags under those locks; validate quantities, address and confirmed prices.

3. Compute exact Decimal total, require buyer balance >= total, aggregate seller credits and buyer debit per account (self-purchase netting included).

4. Insert Orders with delivery snapshot and OrderItems with product-name/price snapshots. Apply account balance deltas and stock decrements. Delete only this buyer's checked-out cart rows. Commit all changes together.

5. Any business error or SQL failure rolls back EVERYTHING. SERIALIZABLE is already configured by the skeleton. Catch SQLSTATE 40001 (serialization) and 40P01 (deadlock) outside the transaction; retry the entire transaction a bounded number of times and then return a retry message. Re-read state and confirmation on every attempt.

6. Serialize duplicate checkout requests so the second sees an empty cart and cannot charge twice. Disable repeated submission in the UI as extra usability protection, not the correctness mechanism. A future idempotency token could also recover the original order after a lost response.

6. Integration agreements needed

Users: balance/address names, default balance zero, safe top-up/withdrawal, purchase-history migration.

Products: global price and availability rules, product id, detail route, form carrying seller/product/quantity.

Sellers: Inventory composite key, inactive removal policy, shared checkout lock ordering, fulfillment ownership and timestamp rule.

Carts: owns CartItems/Orders/OrderItems, checkout transaction and buyer-facing detail. Team agrees on schema before merging shared migration. Social reviews are only in scope for an approved five-person team and are not assumed here.

7. Acceptance checklist and current status

Environment verification: the user ran `setup_course_container.sh` successfully in the existing course Docker environment (Linux, Python 3.12.3, PostgreSQL client 16.15, Poetry 1.8.2). The supplied log confirms schema creation and seed loading into the independent `mini_amazon_carts_dev` database. Flask started on port 8080, and the user confirmed the login/cart browser walkthrough worked. Six isolated SQLite read-path tests also passed. These results do not establish completion of the separate wishlist tutorial, restart-persistence testing or future checkout tests.

Milestone 2: review this design with team; merge all roles into one report; fill README members and truthful progress; personally run skeleton and complete tutorial. The assistant reading TUTORIAL.md is not the student's completion.

Starter: cart SQL model, authenticated HTML/JSON routes, navigation, empty/unavailable states, additive SQL proposal and optional demo fixture. Not implemented: cart mutations, product-detail integration, checkout, order-detail route, purchase-history migration, seller fulfillment.

Validation targets: guest protection; different users cannot see each other's carts; two sellers for the same product remain separate; exact totals; empty cart; inactive/insufficient inventory. Final checkout testing must cover all-or-nothing rollback, insufficient funds, changed prices, simultaneous buyers for the last item, concurrent cart edits, duplicate checkout, unauthorized order viewing and fulfillment.

8. Relationship diagram

Core buyer and listing paths. All arrows point from the referenced parent to its child. Remaining references are listed below to keep the drawing readable.



Additional references: Inventory.seller_id -> Users.id; CartItems.buyer_id -> Users.id; OrderItems.seller_id -> Users.id; OrderItems.product_id -> Products.id. CartItems references the composite Inventory key.

* An order must have at least one line, enforced by the checkout transaction. Each parent may otherwise have zero child rows. Grace owns CartItems, Orders and OrderItems; William owns fulfillment updates.

An empty cart is zero CartItems rows. Order totals and overall fulfillment status are derived. Historical order prices and delivery information are snapshots.

9. Preliminary page wireframes

Proposed final behavior for milestone 2 design review. Controls shown below are planned; only the read-only cart page is currently implemented.

A. Cart /cart

Product | Seller | Quantity input | Current unit price | Subtotal

[Update quantity] [Remove line] Availability per line

Total: SUM(quantity x current price) [Continue shopping] [Review checkout]

Empty: show shopping link. Invalid quantity: preserve prior value. Inactive or insufficient stock: label the affected line; checkout revalidates.

B. Checkout confirmation /checkout

Review all items and latest prices | Total | Available account balance

Recipient and address [Back to cart] [Confirm order]

Insufficient stock/funds or missing address: show error, keep cart. Changed price: refresh summary and require a new confirmation. Success: redirect to order detail.

C. Order detail /orders/<order_id>

Order number | Date | Overall fulfillment | Delivery snapshot

Product | Seller | Purchased quantity | Final unit price | Subtotal | Fulfilled at / Pending

Final total [Purchase history] [Continue shopping]

Buyer-only read access; no edit controls. Each seller fulfills only their lines from the separate seller page. All fulfilled lines make the whole order fulfilled.

Connections and access

Product detail adds a seller-specific quantity to the cart. Purchase history links to the buyer-only order detail. Login precedes private pages; authenticated mutation requests require CSRF protection. Another buyer cannot read the order.